

> **Ficha Prática Nº9 (React – Estados e Interação com Utilizador)**

Nas últimas aulas práticas, foram introduzidos dois conceitos essenciais à criação de aplicações em React: **componentes** e **props** (propriedades). Esta ficha, introduz outro conceito fundamental na criação de uma aplicação em React, designado como **states – estados**, que serão aplicados no jogo de memória.

O conceito de **estado** é um conceito chave em React. Um estado permite armazenar valores de propriedades que pertencem ao componente e quando existe uma alteração dos respetivos valores, é reavaliado de forma a agendar e desencadear uma novamente renderização. Assim, sempre que se pretende associar conteúdo visível na aplicação a uma variável, de forma a que quando se altera essa variável isso seja refletido também na aplicação obrigando a uma novamente renderização, é necessário recorrer a uma **variável de estado**.

Na realidade, uma das grandes tarefas de um programador *front-end* em React, consiste em implementar uma boa *gestão de estados da aplicação*. Dependendo das características da aplicação, esta gestão é variável, podendo ter uma complexidade elevada em aplicações de maior dimensão o que leva, muitas vezes, a recorrer a bibliotecas externas que ajudem e facilitem essa tarefa.

Como referido anteriormente, a implementação de componentes em React pode ser efetuada recorrendo a **componentes de classe** ou **componentes funcionais**. Aquando do lançamento da primeira versão do React, os componentes funcionais, eram bastante limitados, designados de *stateless components*, e como tal, para a criação de aplicações e componentes mais complexos, que envolvessem estados e reutilização da lógica do componente, entre outras características, era necessário recorrer a componentes de classes. No entanto, atualmente o que essencialmente distingue estes dois métodos é a sua sintaxe, mais propriamente, a forma de declaração de um componente, a passagem de propriedades, a manipulação de estados e gestão do ciclo de vida do componente.

Na realidade, o surgimento dos **Hooks**, integrados a partir da versão 16.8 do React em 2018, possibilitou que componentes funcionais pudessem incluir o conceito de **estados**, gestão do ciclo de vida, e outras funcionalidades, que até então, só eram possíveis com recurso a componentes de classe. Esta evolução possibilita implementar uma aplicação complexa, na sua totalidade, recorrendo apenas a componentes funcionais.

Apesar dos hooks apresentarem algumas vantagens como simplificação de escrita e compreensão de código, quando comparado com componentes de classe, ou mesmo no contexto de deixar de ser, de todo, necessário o uso da palavra-chave **this**, que tem comportamento diferente em JavaScript quando comparado com outras linguagens e assim reduzir a confusão que esta possa causar quando aplicada em React; continua a ser possível

a implementar componentes de classe. A escolha é uma opção de implementação, tendo em consideração as características e complexidade da aplicação ou mesmo preferências de cada programador. Portanto, os Hooks possibilitam funcionalidades adicionais a componentes funcionais e não substituem, pelo menos até ao momento, componentes de classe, que ainda apresentam algumas características que não podem ser implementadas apenas com componentes funcionais.

A secção **Hooks:useState**, apresenta alguns aspetos obrigatórios a observar na implementação no React. Para além do hook `useState`, será também introduzido nesta ficha, a manipulação de eventos em React, que se encontra detalhado na secção **Eventos**.

> **Hooks: useState**

Os **Hooks** são **funções** JavaScript, reutilizáveis, com duas regras importantes (obrigatórias!) a ter em consideração quando se pretende utilizar numa aplicação React:

- **devem ser declarados no topo** do componente e não em funções regulares de JavaScript;
- **não devem ser** usados em condições, ciclos ou funções "aninhadas".

O React já disponibiliza um conjunto de **built-in Hooks**, no entanto, é possível também criar os próprios Hooks (<https://react.dev/reference/react/hooks>).

No contexto desta ficha, será usado o Hook **useState**, que, como o próprio nome indica, permite criar variáveis de estado em componentes funcionais. Para além deste, existem outros Hooks, com características e funções diferentes, sendo facilmente distinguíveis pois também iniciam com o prefixo **use**, estando, no entanto, fora do âmbito desta ficha.

O Hook **useState** retorna um *array* de dois valores:

- O primeiro elemento do array refere-se ao valor da **variável de estado**;
- o segundo elemento é uma **referência da função** que permite alterar o valor do estado, dessa variável.

Ao criar a variável de estado com o `useState`, pode-se usar o conceito de **desestruturação de arrays**, existente a partir do ES6, que permite armazenar os valores obtidos pelo `useState` diretamente para variáveis, como se apresenta no código seguinte:

```
const [estado, setEstado] = useState(estadoInicial);
```

Assim:

- a variável **estado** é inicializada com o valor especificado em *estadoInicial*
 - função **setEstado** é o nome dado à função que permite alterar o valor do estado dessa variável.
- Seguindo a convenção, é comum especificar a função com o prefixo *set*, seguido do nome da variável.

Quando ocorre o primeiro render, o valor da variável de estado usado para a aplicação é o estado especificado como argumento, aquando a sua criação (*estadoInicial*). Quando o valor da variável de estado se altera é então agendada uma nova renderização, de forma a que essa alteração se reflita na interface (UI) da aplicação.

Note que as variáveis regulares não irão atualizar o UI. O recurso ao Hook `useState` requer efetuar o *named import* do `useState` da biblioteca do React, como apresentado abaixo, caso contrário, não será possível a sua utilização.

```
import {useState} from "react";
```

> Eventos em React

A **manipulação de eventos** em React é semelhante à manipulação de eventos em elementos DOM, com algumas diferenças na sintaxe:

- Deve-se usar camelCase na especificação do evento;
- Em JSX não é passada uma string, mas sim a função que manipula o evento;
- Não é possível retornar falso, para evitar o comportamento padrão em React. É necessário invocar de forma explícita o *preventDefault*.

Em React, não é necessário recorrer a *addEventListener* para adicionar *listeners* a um elemento DOM após este ser criado. Em vez disso, usam-se, normalmente, atributos de eventos do React como *onClick*, *onChange*, entre outros, diretamente nos elementos JSX, a que se dá o nome de *event handling*. Apenas se especifica um “ouvinte” quando o elemento for renderizado inicialmente, habitualmente no elemento App. A função de manipulação de eventos, bem como outras funções devem ser passadas como *props* para os componentes filhos. Quando se pretende passar um parâmetro para um manipulador de evento ou um *callback* pode-se recorrer a uma *arrow function* no qual envolve o manipulador de eventos e passar parâmetros, ou então recorrendo a atributos *data*.



Em React, a renderização é baseada no estado (state) e nas propriedades (props) dos componentes. O acesso aos elementos DOM de forma direta, por exemplo, com recurso aos métodos *getElementById()* ou *querySelector()*, pode comprometer a renderização da interface, podendo originar problemas de sincronização, diminuição do desempenho, entre outros. Assim, ao utilizar os métodos referidos está a contornar-se, incorretamente, o ciclo de vida dos componentes React. Contudo, importa referir que existem situações pontuais em que continua a ser necessário a utilização dos métodos anteriormente referidos (como exemplo no uso de Portais em React). No entanto, este tema está fora do âmbito desta UC.

Nas secções seguintes, apresentam-se exemplos práticos de manipulação de eventos em React.

> Exemplos Práticos com Variáveis de Estado e Eventos

A Figura 1, recorre à invocação do componente **InfoComponent**, uma para apresentar a descrição associada ao React e outra para a descrição do JavaScript. Além disso, contém ligações que permitem apresentar toda a descrição associada ao logotipo ou, caso contrário, esconder parte dele.

Para implementar este componente, foi necessário especificar uma variável de estado, a que se deu o nome de **isAllVisible**. O comportamento pretendido obtém-se quando se clica respetivamente no texto “Ler mais” / “Reduzir”. Neste exemplo, tanto a variável de estado como o evento estão especificados no componente. De seguida, apresenta-se o código para que o componente tenha este comportamento.

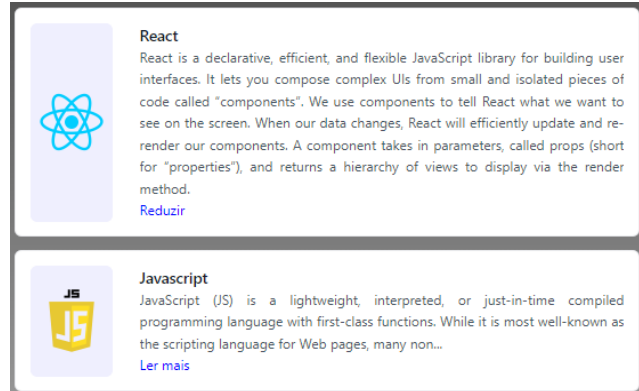


Figura 1 - Exemplo de uso de variável de estado e evento click

```
import React, {useState} from "react";
const maxLength = 200;
function getText(txt, maxLength, showAll) {
  let text;
  if (showAll || txt.length <= maxLength) text = txt;
  else text = `${txt.substring(0, maxLength).trim()}...`;
  return text;
}
function InfoComponent(props) {
  const [isAllVisible, setIsAllVisible] = useState(false);
  const texto = props.children;
  return (
    <div className="wrapper">
      <div className="logo"> <img src={props.src} alt={props.title} /> </div>
      <div className="text">
        <h2>{props.title}</h2>
        <div>
          <p>{getText(texto, maxLength, isAllVisible)}</p>
          {isAllVisible
            ? (<button onClick={()=>setIsAllVisible(false)}>Reduzir</button>)
            : (<button onClick={()=>setIsAllVisible(true)}>Ler Mais</button>)}
        </div>
      </div>
    </div>
  );
}
```

A invocação do componente pode ser efetuada da seguinte forma.

```
<InfoComponent title="React" src="./assets/images/react.png">
React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It lets
you compose complex UIs from small and isolated pieces of code called "components". We use components to
tell React what we want to see on the screen. When our data changes, React will efficiently update and
re-render our components. A component takes in parameters, called props (short for "properties"), and
returns a hierarchy of views to display via the render method.</InfoComponent>
<InfoComponent title="Javascript" src="./assets/images/javascript.png">
JavaScript (JS) is a lightweight, interpreted, or just-in-time compiled programming language with first-
class functions. While it is most well-known as the scripting language for Web pages, many non-browser
environments also use it, such as Node.js, Apache CouchDB and Adobe Acrobat. JavaScript is a prototype-
based, multi-paradigm, single-threaded, dynamic language, supporting object-oriented, imperative, and
declarative (e.g. functional programming) styles. Read more about JavaScript.</InfoComponent>
```

No exemplo seguinte, foram efetuadas alterações de forma a que a variável de estado esteja no componente pai, de forma a que, ao **expandir** o texto, expande todos, assim como, ao **reduzir**, contrai todos.

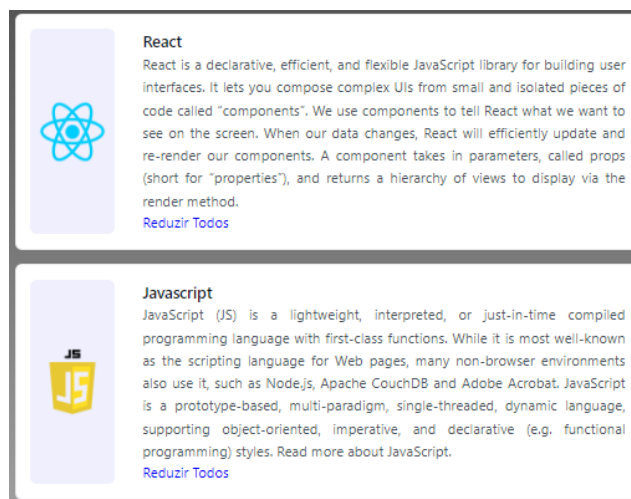


Figura 2 - Uso de variável de estado no elemento Pai

```
function App () {
  const [allTextVisible, setAllTextVisible] = useState(false);
  const handleAllTextVisible = () => setAllTextVisible(!allTextVisible);
  return (<div id="container">
    <InfoComponent title="React" src="./assets/images/react.png"
      allTextVisible={allTextVisible}
      onAllTextVisible={handleAllTextVisible} > React is a declarative...
    </InfoComponent>
    <InfoComponent title="Javascript" src="./assets/images/javascript.png"
      allTextVisible={allTextVisible}
      onAllTextVisible={handleAllTextVisible} > JavaScript (JS) is a light...
    </InfoComponent>
  </div>
);
}
```

```
function InfoComponent(props) {
  const { title, src, allTextVisible, onAllTextVisible } = props;
  const texto = props.children;
  return (
    <div className="wrapper">
      <div className="logo">
        <img src={src} alt={title} />
      </div>
      <div className="text">
        <h2>{title}</h2>
        <p>{getText(texto, maxLength, allTextVisible)}</p>
        {allTextVisible
          ? (<button onClick={onAllTextVisible}> Reduzir Todos </button>)
          : (<button onClick={onAllTextVisible}> Expandir Todos </button>)
        }
      </div>
    </div>
  );
}
```

> Preparação do ambiente

a. Descompacte o ficheiro **ficha9.zip** e inicie o *Visual Studio Code*.

b. Abra a pasta de trabalho no *workspace*.



Os alunos que concluíram a resolução da ficha anterior, devem continuar nesse trabalho.

Os restantes alunos podem resolver esta ficha prática, tendo como base o código fornecido juntamente com esta ficha (Ficha 9).

c. No terminal, digite os seguintes comandos:

→ **npm install** (para instalar as dependências em falta)

→ **npm start** (para iniciar a aplicação)

d. Confirme a aplicação no browser, no endereço <http://localhost:3000/>

Parte I – Exercício: Variável de Estado e Eventos

Pretende-se que o aluno implemente a apresentação dos elementos do painel de controlo, ativando e desativando elementos, quando o jogo se inicia. As figuras abaixo apresentam o comportamento desejado.



Nota: Os alunos que recorreram ao projeto disponibilizado nesta ficha, as peças de jogo não serão apresentadas (implemente essa parte no fim desta ficha, consultando a ficha prática 8).

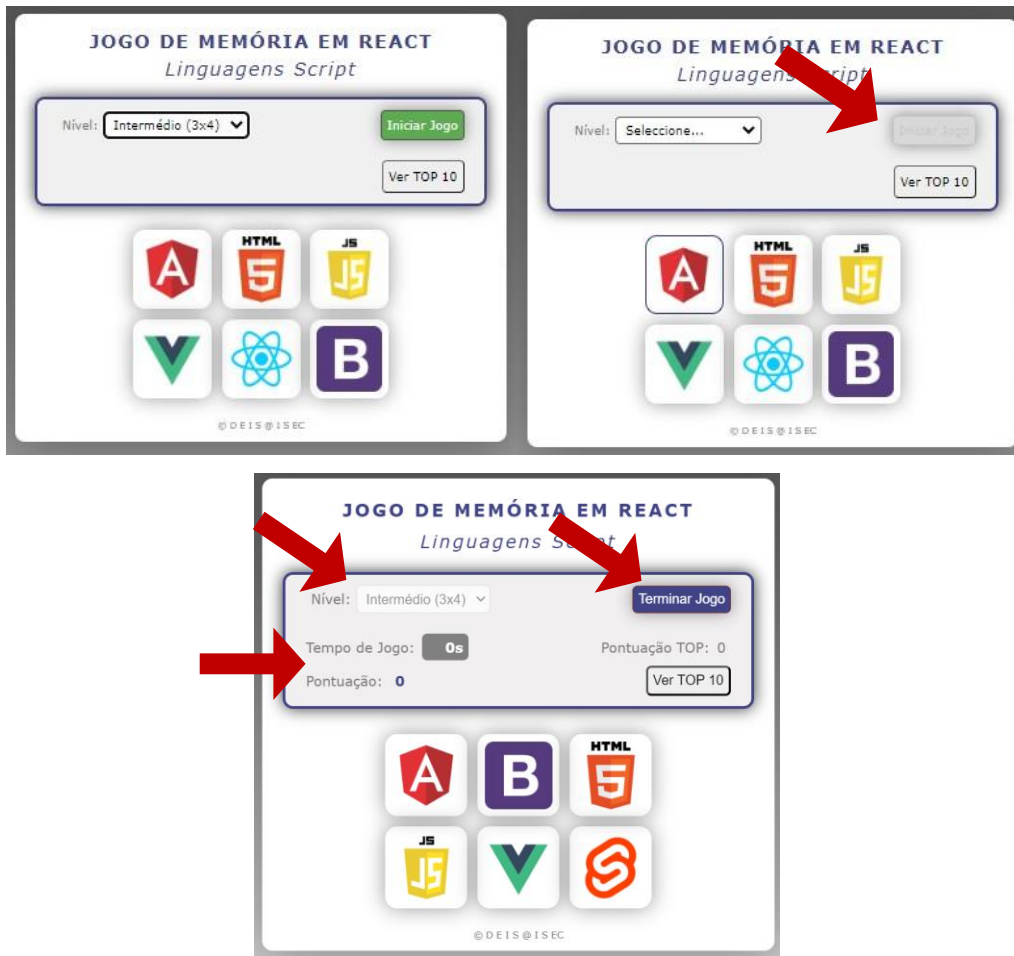


Figura 3 - Jogo de Memória (estados e eventos) – Ficha 9

1> Implemente os seguintes passos no componente **App**:

a. Declare e inicialize as seguintes variáveis de estado:

- `gameStarted` (estado inicial deve ser `false`)
- `selectedLevel` (estado inicial deve ser `"0"`)

b. Implemente a função `handleGameStart` que deve **alterar** o **valor da variável de estado** que gere o início do jogo. Se esta for **true**, deve mudar para **false** e vice-versa.

c. Implemente a função `handleLevelChange` que deve **alterar** o **valor da variável de estado** que gere o nível do jogo. Isto é, esta função, deverá armazenar o nível de jogo selecionado.

Nota: A identificação do alvo atual para o evento pode ser especificada através da propriedade `currentTarget`, referenciado assim o elemento associado ao *event handler*.

d. Altere a invocação do **ControlPanel**, especificando os atributos seguintes:

- `gameStarted={gameStarted}`
- `onGameStart={handleGameStart}`
- `selectedLevel={selectedLevel}`
- `onLevelChange={handleLevelChange}`

2> No componente **ControlPanel** (ficheiro `control-panel.componente.jsx`), implemente os seguintes passos:

a. Receba as propriedades do componente (**props**), com recurso à desestruturação de *arrays*, inicializando as seguintes variáveis:

- `gameStarted`
- `selectedLevel`
- `onGameStart`
- `onLevelChange`

b. No componente `select`, especifique os atributos `disabled` e `onChange`, com os valores/referências obtidas pelas props.

c. No componente `button`, especifique os atributos `disabled` e `onClick`, com os valores/referências obtidos pelas props. Além disso, especifique corretamente o código para que o texto do botão seja alterado para “Terminar Jogo” ou “Iniciar Jogo” dependendo do estado do Jogo.

d. Por fim, adicione a class `gameStarted` aos elementos `dl` que envolvem o tempo de jogo, Pontuação TOP e Pontuação, apenas quando o jogo já tenha iniciado de forma a que estes elementos fiquem visíveis. Implemente este comportamento, com o menor código possível.